

Kite Algo Trader — Interview Prep

In This Edition

1	1. The 30-second pitch	2
2	2. Tech stack (and why each)	2
3	3. Architecture in six boxes	2
4	4. Problems faced & how we solved them (the part interviewers care about)	3
5	5. Likely questions → answers	4
6	6. "What would you do differently"	5

Personal notes. Untracked on purpose — do **not** commit.

1 1. The 30-second pitch

"I built a multi-strategy algorithmic trading platform for Indian equities, index options, and MCX commodities — ~31 strategies running concurrently, paper and live (via Zerodha). It ingests real-time market data over WebSockets, runs a paper/live order engine, has a full observability stack, and a backtesting engine that replays the *real* strategy code. It runs 24/7 on two cloud VMs. The interesting parts weren't the strategies — they were keeping a stateful, real-time, money-handling system *reliable*: crash recovery, data-feed failures, resource limits, race conditions."

Lead with "**reliability of a stateful real-time system**" — that's what makes it senior-level, not "I wrote trading strategies."

2 2. Tech stack (and why each)

Layer	Tech	Why
Backend	Python, Flask, Flask-SocketIO, eventlet	Real-time push to UI; async I/O on one process
Data feeds	Angel One SmartAPI (WS+REST, primary), Dhan (WS option ticks), yfinance (fallback)	Multi-source with fallback behind one facade
Execution	KiteConnect/Zerodha (live orders + OAuth), custom PaperEngine	Paper-first, selective live
Storage	SQLite (WAL) for live state/trades, DuckDB + Parquet for analytics/cold tier	Lightweight OLTP + columnar OLAP, no DB server
Frontend	Vanilla JS SPA, Socket.IO, PWA/service worker , Lightweight Charts	Mobile-installable, no framework overhead
Observab	VictoriaMetrics , Grafana , Loki , process-exporter/Alloy, GlitchTip (Sentry-compatible)	Full RED monitoring across 2 hosts
Quant	Pure-Python indicators (EMA/ATR/RSI/ADX/SuperTrend/VWAP), Black-Scholes greeks, QuantLib IV , pandera , Optuna	Backtest optimization + data validation
Infra/Dev	Oracle Cloud (1 GB micro + 24 GB ARM), systemd units+timers, nginx TLS, OCI Object Storage backups, GitHub Actions CI (ruff, pytest, gitleaks, pip-audit, Dependabot)	Free-tier-constrained, automated

3 3. Architecture in six boxes

Data feeds $\rightarrow$ MarketDataService (caching facade) $\rightarrow$ 31 Strategy threads (BaseStrategy ABC) $\rightarrow$ OrderManager $\rightarrow$ Paper/Live engine $\rightarrow$ SQLite, with three cross-cutting safety systems: **Watchdog** (detects stuck strategies), **TradeResumer** (restores open trades after restart), **Reconciler** (detects position $\leftrightarrow$ state drift). Plus a **backtest engine** that replays the real strategy code, and the **observability stack**.

4. Problems faced & how we solved them (the part interviewers care about)

(1) Duplicate orders / orphaned positions (thread-lifecycle race)

- › Symptom: a strategy bought the same option 4× in 60s; positions accumulated; legs left open after close → hidden losses.
- › Root cause: the watchdog recycled a "stuck" strategy with `stop()`; `start()`, but `stop()` only flipped a flag — never **joined** the thread. If the strategy was hung on a blocking call (when it looks stuck), the old thread was alive, so `start()` spawned a SECOND thread. Two threads each fired entries.
- › Fix: `start()` refuses to spawn while the old thread is alive; two-phase recycle (`stop` → `drain` → `restart`); 15:25 EOD square-off net.
- › Lesson: a kill that doesn't join is not a kill. Thread lifecycle + idempotency.

(2) Watchdog killing healthy strategies (liveness vs progress)

- › Symptom: strategies recycled every few minutes, then daily ~14:54.
- › Root cause: watchdog recycles anything whose heartbeat is stale; idle/wait loops used plain `time.sleep()` → no heartbeat → looked hung.
- › Fix: heartbeat-aware `self._sleep()` — beats every 30s but stays interruptible, so a GENUINE hang (stuck outside the sleep) is still caught.
- › Lesson: distinguish "alive" from "making progress"; the liveness signal must come from the work loop.

(3) Expired options stranded forever (don't gate exits on data)

- › Symptom: a 16-Jun Iron Condor still held on 17-Jun.
- › Root cause: exit computed AFTER an LTP fetch — `if net_premium < 0: retry`. Expired options stop ticking → premium permanently unavailable → retried forever, never exited.
- › Fix: expiry-date safety net (settle at intrinsic-vs-spot, no LTP); state-independent sweep that parses expiry from the position symbol; paper engine accepts ₹0 settlement only for a genuine expired close.
- › Lesson: never gate a critical exit behind data that can disappear.

(4) ₹33k phantom loss from a fabricated price

- › Root cause: when live price was missing, the close used a hard-coded or 0.5 fallback fill.
- › Fix: never fabricate — retry, then skip; settle at intrinsic or not at all.
- › Lesson: in a money system, a WRONG number is more dangerous than NO number.

(5) The 1 GB VM core-dumping (SIGBUS)

- › Root cause: DuckDB default memory ~80% of RAM; concurrent with the trader on 1 GB → over-commit, swap-thrash, page fault on an **mmap'd Parquet** page → SIGBUS, killing the whole process.
- › Fix: cap DuckDB memory/threads + disk-spill temp dir (env-overrideable); also disabled SQLite mmap (same SIGBUS-under-concurrent-write mode). `systemd Restart=on-failure` as backstop.
- › Lesson: resource limits in constrained envs; why mmap faults become SIGBUS.

(6) NSE 403-blocking cloud IPs (vendor reliability)

- › Root cause: NSE blocks data-center IPs; feed silently went stale on VMs (worked locally).
- › Fix: migrated ALL live data to Angel One (incl. VIX), gated dead NSE behind a flag. The `MarketDataService` facade meant 31 strategies didn't change.

- › Lesson: abstract vendors behind a facade; don't trust one external dependency.

(7) API rate-limit storms

- › Root cause: Angel One throttled (~1,400 hits/host/day); candle fetches returned empty → strategies blind.
- › Fix: caching (dedup fetches) + process-wide throttle (3 req/s) + exponential backoff (5→10→20→40s) + alerting. ~90% fewer hits.
- › Lesson: caching + throttling + backoff as the standard rate-limit pattern.

(8) eventlet broke SSL libs (async monkey-patching pitfall)

- › Root cause: eventlet monkey-patches the SSL stack; Telegram/yfinance recursed infinitely.
- › Fix: pull the original un-patched `urllib.request` via `eventlet.patcher.original()` for those calls.
- › Lesson: know your concurrency model's failure modes; monkey-patching is leaky.

(9) Crash recovery & state drift

- › Problem: a restart could lose in-memory state while positions stayed open → strategy went blind, re-entered, accumulated orphans.
- › Fix: TradeResumer restores open trades on boot; Reconciler detects "position without trade-state" drift; re-entry guard refuses a new position-group while untracked legs are held.
- › Lesson: stateful systems need explicit crash-recovery + a reconciliation loop.

(10) A refactor silently broke DB reads

- › Symptom: History/P&L tabs went blank after a code reorg.
- › Root cause: moving files into deeper packages broke `__file__`-relative path math → DB path resolved to an empty file. Tests masked it (override the path); boot didn't error (empty DB returns no rows).
- › Fix: corrected path math + regression tests pinning every `__file__`-derived path to repo root.
- › Lesson: depth-changing refactors are sneaky; "it imports and boots" ≠ "it works."

□ Backtests that didn't match live

- › Problem: original backtester used per-strategy adapters → diverged from live.
- › Fix: rebuilt to replay the REAL strategy code against recorded data, with a data-fidelity grade, Optuna optimization + walk-forward, as a niced subprocess so it never starves the live trader.
- › Lesson: backtest-live parity — different code = the backtest is lying.

5 5. Likely questions → answers

- › **Hardest bug?** → (1) duplicate orders: observability → two threads → `stop()` never joined → spawn guard + two-phase recycle. Root cause, not symptom; killed a class.
- › **Reliable long-running system?** → detection (watchdog/heartbeat), recovery (crash-restart + persisted state, resume not re-fire), reconciliation (drift loop), systemd floor. Each added after a real incident.
- › **Unreliable external data?** → facade + multi-source fallback + cache/throttle/backoff + "no data is a first-class state, never fabricate." NSE swap touched 1 module not 31.
- › **Testing vs live markets?** → paper engine mirrors live path; backtest replays real code; parity tests (old impls as oracles, bit-for-bit); unit tests on money paths.

- › **Resource constraints?** → 1 GB VM: DuckDB caps + hot/cold Parquet tiering + tpool offload; same code scales to 24 GB via env vars.
- › **Observability?** → metrics+Grafana+Loki+GlitchTip, RED dashboards, per-strategy P&L, data-source freshness, config-change tracking. Most debugging started from a dashboard.
- › **Decision you'd revisit?** → SQLite+eventlet single-process was right for solo free-tier, but caused SIGBUS/concurrency + monkey-patch pitfalls; at scale → Postgres + real task model. Also front-load tests.
- › **Ship safely?** → one-command multi-host deploy (ff-only + compile-check + restart + health-check), CI every push, DB changes after close; hardened deploy to fail loudly after a silent no-op.
- › **Concurrency model?** → eventlet (green threads) for I/O + Socket.IO, OS threads per strategy; bugs lived at that boundary (join semantics, heartbeat starvation, SSL monkey-patch).

6. "What would you do differently"

1. Storage: SQLite was right for solo/free-tier but hit SIGBUS/concurrency walls; Postgres removes a bug class — simplicity vs concurrency-safety tradeoff.
2. Tests first: best fixes ended with "...and added the test that would've caught it" (phantom-fill, path-depth) — both invisible to "it runs."

Closing framing: "Solo project, but I treated it like production — because it trades real money. Most of the engineering went into the boring-but-hard parts: not crashing, not double-trading, not trusting a feed, and being able to see what it's doing."